THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

Automated DevOps workflow for deploying a containerized web application using Docker, Kubernetes, Helm, NGINX, and GitLab CI/CD.

Project Description:

This project demonstrates a fully automated Complete DevOps workflow for deploying a containerized web application using GitLab CI/CD, Docker, Kubernetes, Helm, and NGINX. The workflow follows a modern DevOps approach to streamline development, testing, and deployment processes, ensuring a seamless and reliable delivery pipeline.

The process starts with the source code hosted in a **GitLab repository**, where any commit or push automatically triggers the GitLab Runner. The Runner builds the Docker image of the application, tags it, and pushes it to the **GitLab Container Registry**. Once the image is available, the pipeline triggers the Kubernetes cluster deployment.

Kubernetes is used to orchestrate containerized workloads efficiently. Helm is utilized for templated deployments, ensuring reproducibility and easy management of complex Kubernetes configurations. NGINX acts as a reverse proxy, allowing access to the deployed web application via a browser.

The project also demonstrates the use of **masked environment variables** and **secure handling of Kubernetes credentials** in GitLab CI/CD, following industry best practices for security and automation. This ensures that sensitive information like kubeconfig or container registry credentials are never exposed while enabling full automation.

This project highlights the power of combining DevOps tools to create a **robust, scalable, and automated deployment pipeline** that mirrors real-world enterprise-level practices.

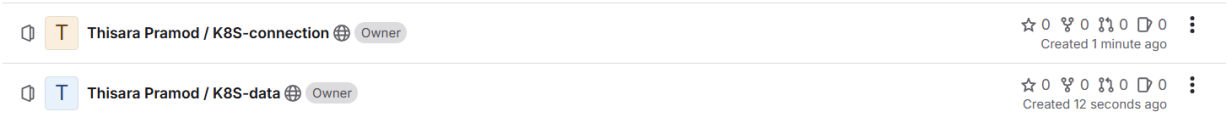
THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- ## Create separate GitLab repositories

  One for Agent connections, other for CI/CD workflow

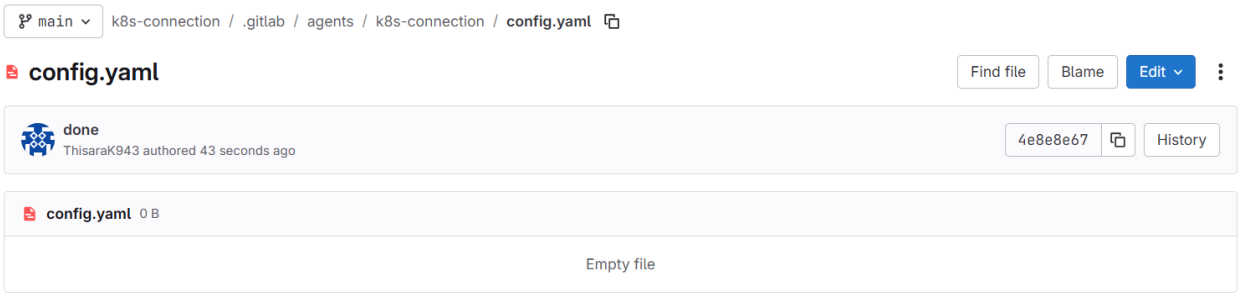


- ## Installing the agent for Kubernetes

  Existing k8s cluster is ready

```
[thisara@localhost ~]$ kubectl get nodes
NAME       STATUS   ROLES           AGE    VERSION
minikube   Ready    control-plane   135m   v1.34.1
[thisara@localhost ~]$
```

- ## Create an agent repo on GitLab

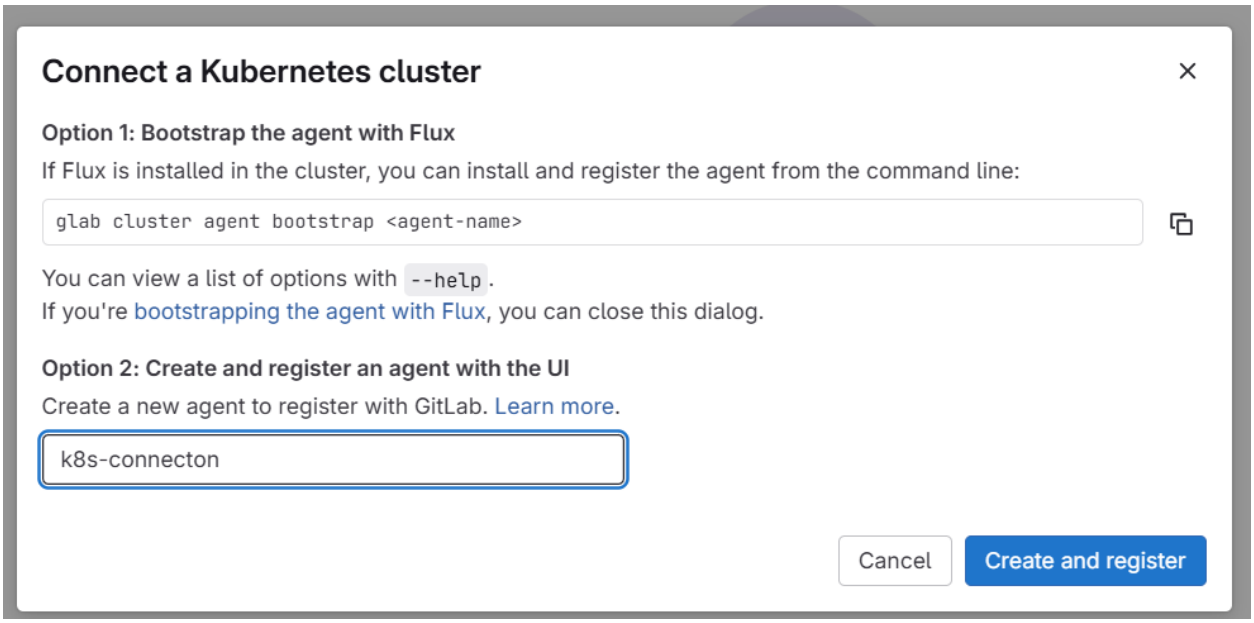


THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- ## Register agent with GitLab



- ## Connect a Kubernetes cluster to GitLab steps

The agent uses the token to connect with the GitLab

```
[thisara@localhost ~]$ helm repo add gitlab https://charts.gitlab.io
"gitlab" has been added to your repositories
[thisara@localhost ~]$
```

```
[thisara@localhost ~]$ helm repo update
Hang tight while we grab the latest from your chart repositories...
...Successfully got an update from the "gitlab" chart repository
Update Complete. ⎈Happy Helming!⎈
[thisara@localhost ~]$
[thisara@localhost ~]$
```

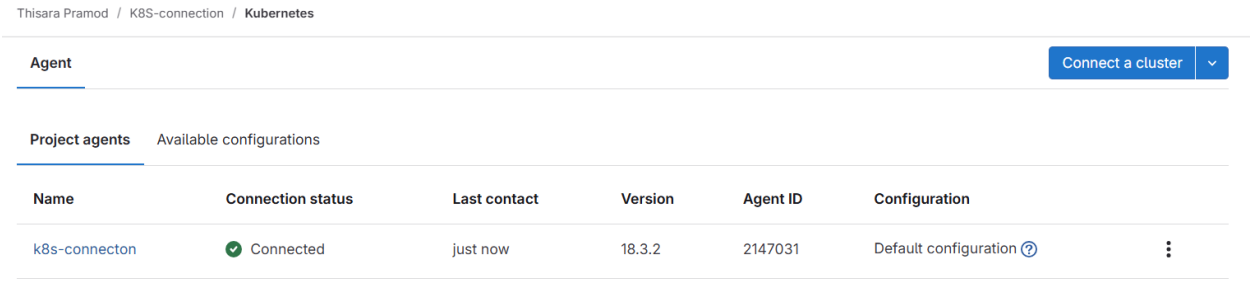
```
[thisara@localhost ~]$ helm upgrade --install k8s-connecton gitlab/gitlab-agent \
>     --namespace gitlab-agent-k8s-connecton \
>     --create-namespace \
>     --set config.token=glagent-sTx4ZkS9aGmEzYzbH-4JNxQhUVXUqeAecjvKCrWjDuC4wR74Vg \
>     --set config.kasAddress=wss://kas.gitlab.com
Release "k8s-connecton" does not exist. Installing it now.
NAME: k8s-connecton
LAST DEPLOYED: Fri Sep 12 03:35:24 2025
NAMESPACE: gitlab-agent-k8s-connecton
STATUS: deployed
REVISION: 1
TEST SUITE: None
NOTES:
```

THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- **The agent is connected**



- **Docker image creation steps**

Docker file

```
[thisara@localhost 1-K8S-CICD]$ cat Dockerfile
FROM nginx:latest
COPY src/. /usr/share/nginx/html

[thisara@localhost 1-K8S-CICD]$
```

I use nginx image as a base image to build docker image in my project

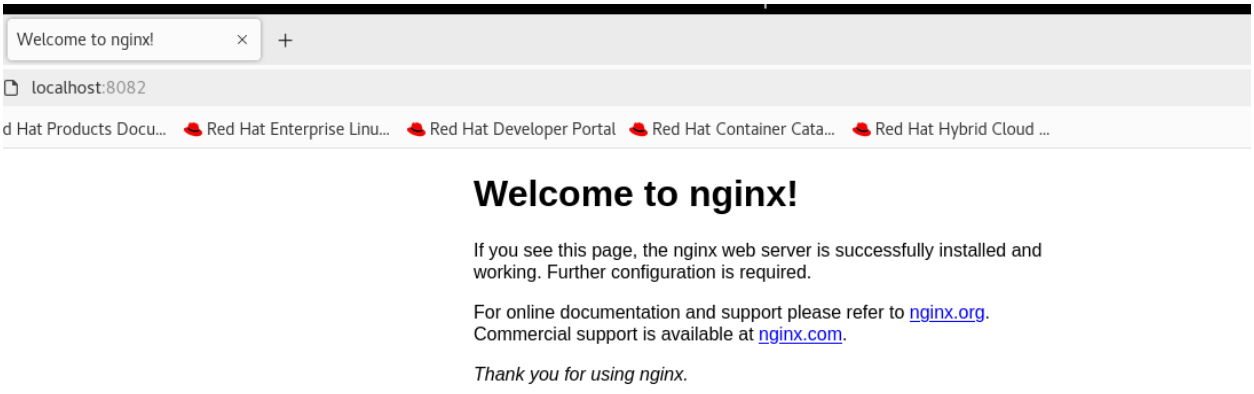
THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- **Check the nginx image locally**

```
[thisara@localhost 1-K8S-CICD]$ docker run -d --name web001 -p 8082:80
"docker run" requires at least 1 argument.
See 'docker run --help'.

Usage:  docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Create and run a new container from an image
[thisara@localhost 1-K8S-CICD]$ docker run -d --name web001 -p 8082:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
d107e437f729: Pull complete
cb497a329a81: Pull complete
f1c4d397f477: Pull complete
f72106e86507: Pull complete
899c83fc198b: Pull complete
a785b80f5a67: Pull complete
6c50e4e0c439: Pull complete
Digest: sha256:d5f28ef21aabddd098f3dbc21fe5b7a7d7a184720bc07da0b6c9b9820e97f25e
Status: Downloaded newer image for nginx:latest
845c32aef595f35060e8e77cb36c5869ef2b4f302e96f32e63db58fb41e1d653
[thisara@localhost 1-K8S-CICD]$
[thisara@localhost 1-K8S-CICD]$
[thisara@localhost 1-K8S-CICD]$
```



THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- **Write a GitLab CI/CD script to build docker image automatically**

Edit Visualize Validate NEW Full configuration

CI/CD Catalog Help Editor accessibility guide

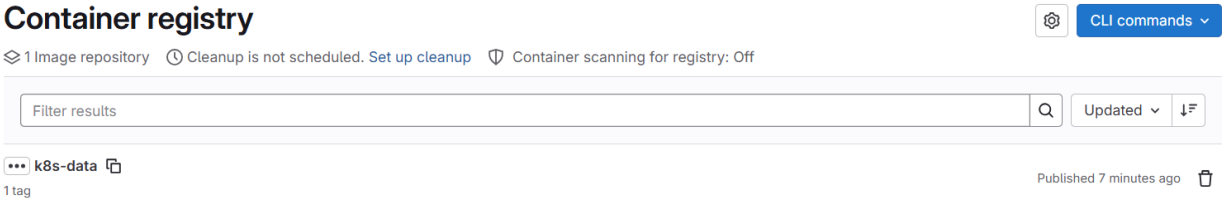
```yaml
stages:
  - build

build_image:
  image: docker:latest
  stage: build
  services:
    - docker:dind
  variables:
    DOCKER_DRIVER: overlay2
  script:
    # Login to GitLab Container Registry
    - echo "$CI_REGISTRY_PASSWORD" | docker login $CI_REGISTRY -u $CI_REGISTRY_USER --password-stdin

    # Build the image
    - docker build -t $CI_REGISTRY/thisarak943/k8s-data:latest .

    # Push the image
    - docker push $CI_REGISTRY/thisarak943/k8s-data:latest

    - echo "✅ Image built and pushed successfully"

```

I using Dind(Docker in Docker) on for GitLab runners, in my .gitlab-ci.yml file

- **After committed the changes pipeline works and build docker image and push it it to GitLab container registry.**

```
111 8e7d6b511078: Pushed
112 36f5f951f60a: Pushed
113 c855abf10cdc: Pushed
114 latest: digest: sha256:69b6c6b6c2f360708b9241b972603b50b9591ce9e8a6ad91fa08807717ebaf5f size: 1988
115 $ echo "✅ Image built and pushed successfully"
116 ✅ Image built and pushed successfully
117 Cleaning up project directory and file based variables                    00:01
118 Job succeeded
```

- **Created Docker Image pushed it to GitLab container registry**



THISARA KANDAGE

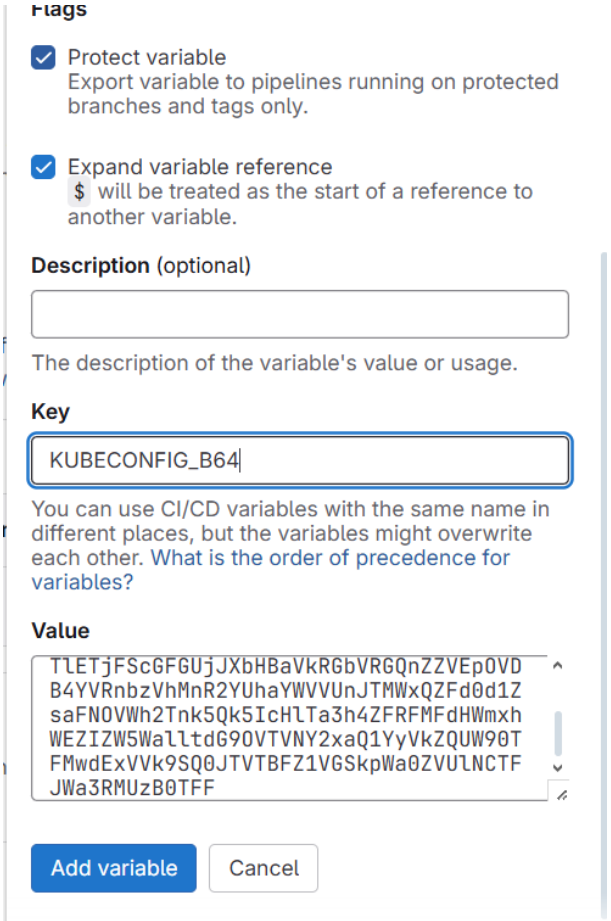
UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

Then the rest thing is, in the Kubernetes cluster how I used this image (already pushed to GitLab container registry)

- **Create variable for deployment**

For that I encoded kubeconfig.yaml file to **base64-encoded format** then use it as a environment variable in GitLab pipeline script.



THISARA KANDAGE

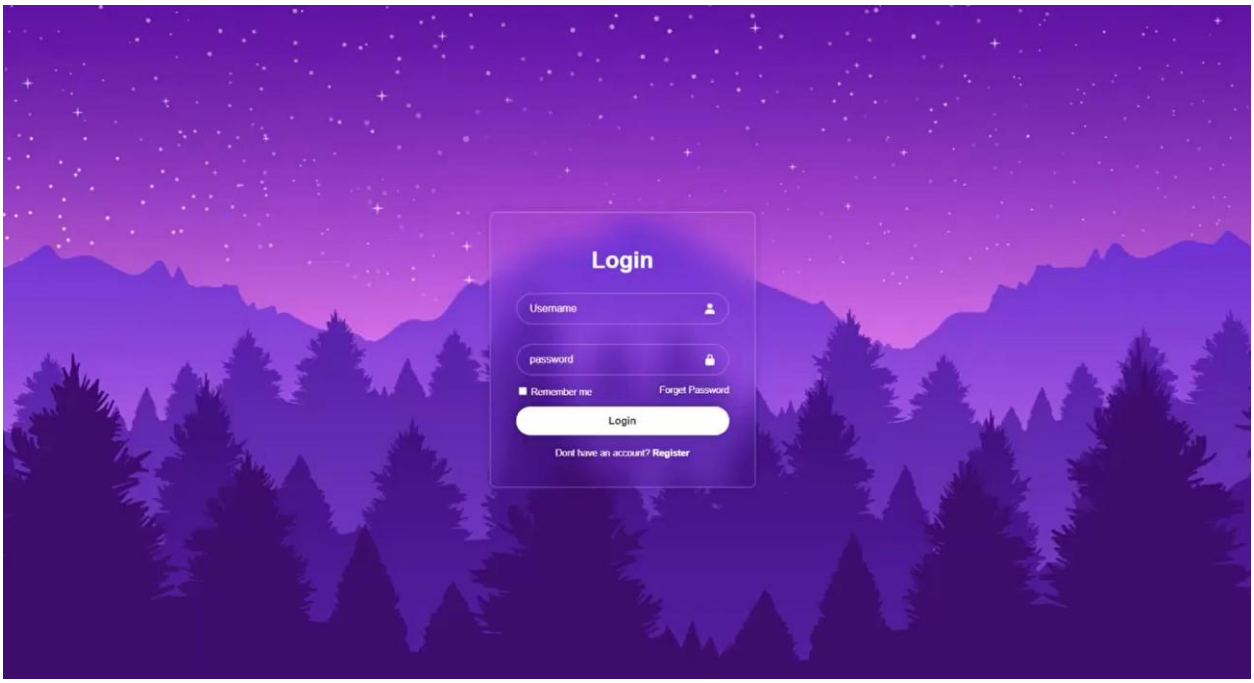
UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

- **Automate the deployment using pipeline script.**

```
29
30  deploy_project:
31    stage: deploy
32    image:
33      name: bitnami/kubectl:latest
34      entrypoint: [""]
35    script:
36      # Decode the kubeconfig safely
37      - echo "$KUBECONFIG_B64" | tr -d '\n' | base64 -d > kubeconfig.yaml
38      - export KUBECONFIG=$(pwd)/kubeconfig.yaml
39
40      # Use the specified Kubernetes context
41      - kubectl config use-context $KUBE_CONTEXT
42
43      # Verify the cluster
44      - kubectl get pods
45      - kubectl get nodes -o wide
46
47
```

- **After running the pipeline access the application on browser**



THISARA KANDAGE

UNDERGRADUATE - SLIIT

E-mail LinkedIn GitHub Website

Project Summary

In this project, I successfully built a fully automated CI/CD pipeline that integrates containerization, orchestration, and deployment using cutting-edge DevOps tools. Key outcomes and learnings include:

1. **Containerization with Docker:** Learned to package web applications into Docker containers for consistent deployment across environments.
2. **Kubernetes Orchestration:** Gained hands-on experience deploying, scaling, and managing applications on Kubernetes clusters.
3. **CI/CD Automation:** Implemented GitLab CI/CD pipelines for automated building, testing, and deployment of Docker images.
4. **Secure DevOps Practices:** Learned to manage sensitive credentials securely using masked variables and environment configurations.
5. **Helm and NGINX Integration:** Used Helm charts for templated deployments and NGINX for reverse proxying and routing traffic.
6. **Real-world DevOps Workflow:** Simulated an end-to-end enterprise deployment pipeline, combining multiple DevOps tools to achieve fully automated deployment.

This project enhanced my understanding of **DevOps principles, automation, container orchestration, and continuous delivery**, preparing me to contribute effectively in a professional DevOps or SRE environment. It serves as a complete demonstration of modern DevOps practices applied to a real-world web application.